

Received 20 June 2025, accepted 2 July 2025, date of publication 8 July 2025, date of current version 25 July 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3587027

RESEARCH ARTICLE

Cross-Filter Structured Pruning for Efficient Sparse CNN Acceleration

NGOC-SON PHAM¹, (Member, IEEE), SANGWON SHIN¹, LEI XU², (Member, IEEE), WEIDONG SHI³, (Senior Member, IEEE), AND TAEWEON SUH¹, (Member, IEEE)

¹Department of Computer Science and Engineering, Korea University, Seoul 02841, South Korea

²Department of Computer Science, Kent State University, Kent, OH 44240, USA

³Department of Computer Science, University of Houston, Houston, TX 77204, USA

Corresponding author: Taeweon Suh (suhtw@korea.ac.kr)

This work was supported in part by the Institute of Information and Communications Technology Planning and Evaluation (IITP) Grant funded by Korea Government [Ministry of Science and Information and Communication Technology (MSIT)] (RISC-V Based System Software Development for Open Ecosystem of SDR) under Grant RS-2024-00459774; and in part by Samsung Research, Samsung Electronics Company Ltd.

ABSTRACT Convolutional Neural Networks (CNNs) are widely used in vision tasks for resource-constrained environments due to their computational efficiency and strong generalization. However, the dominance of 1×1 convolutions in modern CNN architectures introduces challenges for sparsity-aware hardware accelerators, particularly in processing element (PE) load balancing, which limits speedup in sparse inference. To address this issue, this paper proposes a cross-filter structured pruning method, enforcing a uniform sparsity pattern across multiple filters to ensure balanced workload distribution among PEs. This approach is further extended to $k \times k$ convolutions by decomposing them into 1×1 filters, improving applicability across various CNN layers. This paper also proposes an intra-kernel parallelism technique which significantly reduces the size of PE's local buffers, a critical bottleneck in sparse CNN accelerators. Experimental results show that the proposed approach maintains accuracy comparable to globally unstructured pruning while significantly enhancing inference speed. FPGA implementation and cycle-accurate simulations confirm improvements in processing speed, energy efficiency, and hardware utilization, making this method well-suited for edge and mobile AI applications. Specifically, the proposed architecture achieves $1.14\times$ to $1.6\times$ speedup over Sparten for various CNN models and delivers $7.6\times$ and $1.9\times$ higher energy efficiency compared to Sparten and StarSPA, respectively. In terms of area efficiency, synthesis results show a $1.73\times$ – $10.95\times$ reduction in required hardware primitives compared to Sparten and StarSPA.

INDEX TERMS AI accelerator, convolutional neural networks (CNNs), sparsity exploitation, data compression, dataflow, network on a chip (NoC).

I. INTRODUCTION

Although *Vision Transformer* (ViT) models have gained significant attention and achieved remarkable success in computer vision, *Convolutional Neural Networks* (CNNs) still offer distinct advantages, particularly in applications demanding high computational efficiency and compact model size, such as embedded and mobile devices [1], [2]. Furthermore, CNNs exhibit strong generalization capabilities

on medium- to small-scale datasets while delivering fast processing speeds, even achieving real-time performance [3]. The success of CNNs can be attributed in part to their ability to scale in depth, enabling the extraction of multi-level features: from low-level features such as edges and corners in the initial layers to higher-level representations of shapes and object structures in deeper layers [4], [5].

Among the key architectural components, 1×1 convolution (also known as *pointwise convolution*) plays a crucial role in increasing network depth while maintaining computational efficiency [6]. Conventional $k \times k$ filters (also referred to as

The associate editor coordinating the review of this manuscript and approving it for publication was Mario Donato Marino ¹.

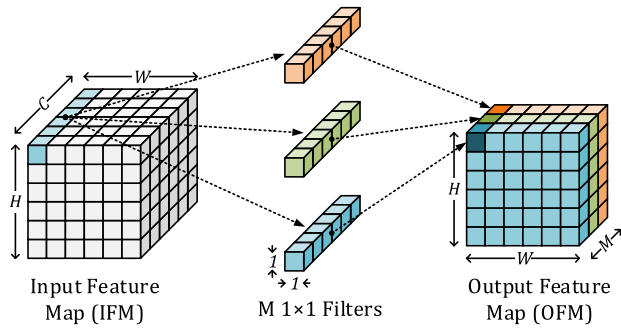


FIGURE 1. Illustration of a pointwise convolution layer.

spatial filters, where $k = 3, 5, 7$, or 11) in CNNs extract local spatial features from a neighborhood in an image. In contrast, a 1×1 filter operates differently: instead of convolving over a spatial region, it processes all channels at a single pixel location. FIGURE 1 illustrates this concept. In CNN architectures, 1×1 convolution primarily facilitates dimensionality reduction by controlling the number of channels in a feature map [7]. Additionally, it effectively performs channel-wise feature transformations while introducing non-linearity into the network. This makes it a powerful tool for designing efficient and accurate architectures with controlled complexity.

Over the years, the proportion of computations involving 1×1 convolutions in CNNs has increased significantly, as shown in TABLE 1. In the early stages of deep learning architectures, 1×1 convolutions were rarely used. However, in later designs, they became an essential component. In fact, the majority of convolutional operations in modern CNNs now involve 1×1 convolutions. For example, in 2014 and 2015, when VGG16 [5] and ResNet34 [8] were introduced, 1×1 convolutions were not yet incorporated. In contrast, by 2017, 98.72% of the computations in MobileNet [2] involved 1×1 convolutions, and this figure increased to 99.03% in ShuffleNet [9] by 2018. This shift was accompanied by a significant increase in network depth and accuracy. Given this trend, hardware accelerator design for CNN inference must prioritize architectural optimizations for efficiently handling 1×1 convolution operations, as they now constitute the dominant computational workload in modern CNN architectures.

In addition to the widespread use of 1×1 filters, another prevalent approach to accelerating CNN inference is leveraging network sparsity [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21], [22], [23], [24], [25], [26], [27], [28], [29]. Since the primary computations in inference involve multiplications and additions, any operation where one of the operands (either *input activation* or *weight*) is zero becomes redundant and can be skipped. Deep learning models are typically initialized with an excessive number of connections [30]. Thus, removing redundant connections (i.e., *exploiting sparsity*) can significantly reduce computational workload while maintaining model accuracy. In some cases, computational demand can be reduced by up

TABLE 1. MAC computation ratio in CNN models.

Year	Model	Number of Layers	Percentage of MACs from 1×1 Convs
2012	AlexNet	8	0 %
2014	VGG16	16	0 %
2015	ResNet34	34	0 %
2014	GoogLeNet	22	25.6 %
2015	ResNet50	50	76.16 %
2017	DenseNet121	121	89.79 %
2017	MobileNetV1	28	98.72 %
2018	ShuffleNet	86	99.03 %
2019	EfficientNetV2_m	48	94.72 %

to 90% [20]. This not only decreases computational complexity, leading to lower energy consumption and faster processing, but also reduces data storage requirements, as sparse matrices can be compressed. Consequently, memory access overhead is minimized, which is crucial since, according to several deep learning studies, memory access energy consumption often exceeds the energy required for inference computation [31].

Sparsity naturally arises in CNNs due to the use of *Rectified linear unit (ReLU)* activation functions [32], [33]. As the most widely used activation function in CNNs, ReLU transforms small negative values into zero while retaining positive values, thereby introducing sparsity in intermediate layers. Additionally, sparsity can be intentionally induced through *filter pruning* techniques, where small-weight values are set to zero, followed by retraining the network. Research has shown that filter sparsity can reach up to 80% without significantly affecting model accuracy [34].

However, without appropriate optimizations in *hardware-algorithm co-design*, sparsity exploitation may not lead to substantial processing speedup. This is primarily due to load imbalance among *processing elements (PEs)* in the accelerator. Given the architectural characteristics of CNNs, *data parallelism* and *data reuse* are typically leveraged to enhance performance and reduce memory access overhead. Specifically, at each CNN layer, multiple filters convolve with an *input feature map (IFM)* simultaneously, enabling parallel computation across multiple PEs. Each PE stores a filter, while the IFM is broadcast to all PEs, facilitating the generation of distinct *output feature map (OFM)* channels. This parallel processing strategy is commonly employed in most modern accelerators such as SCNN [17], GoSPA [19] and StarSPA [23]. However, when weights are sparse, *load imbalance* arises as some PEs become idle while waiting for others to complete their computations. This issue is particularly severe in layers dominated by 1×1 convolutions.

The graph in FIGURE 2 compares the computational performance of the StarSPA [23] architecture when running inference on pruned and unpruned versions of

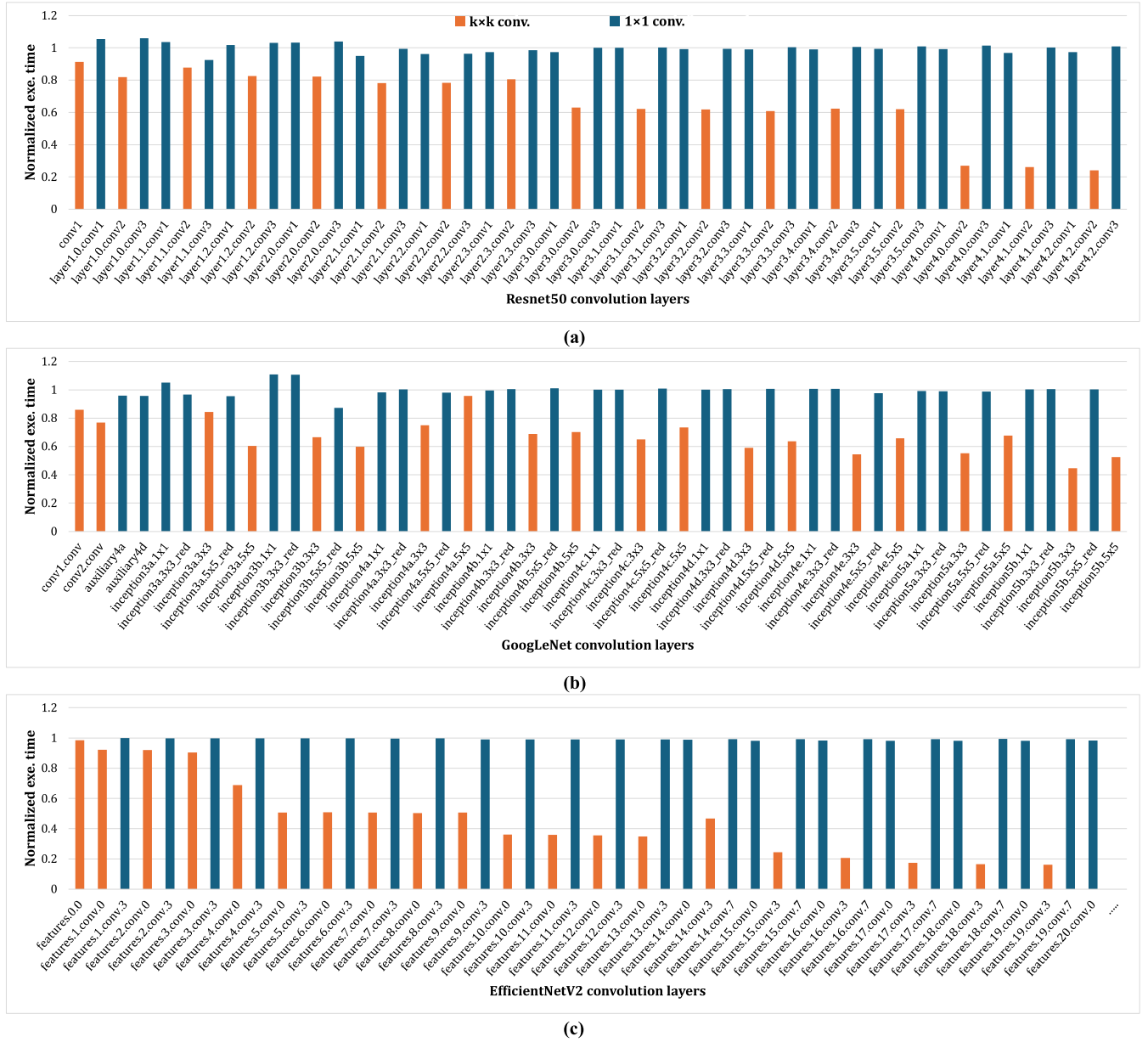


FIGURE 2. Normalized execution time of pruned CNN relative to unpruned CNN for (a) ResNet50, (b) GoogLeNet, and (c) EfficientNetV2.

GoogLeNet [7], ResNet50 [8] and EfficientNetV2 [35]. In the pruned version, 60% of the weights were *globally unstructured* removed (set to zero) [36], whereas in the unpruned version, 100% of weights were nonzero. As analyzed earlier, to minimize memory access overhead, StarSPA—like many accelerator architectures—broadcasts IFMs to each *PE cluster*, where each PE cluster consists of 16 PEs. These PEs operate synchronously, meaning that a new chunk of the IFM is broadcast only when all PEs in a cluster have completed their computations. This synchronization requirement for broadcasting *input activations (iacts)* exacerbates workload imbalance among PEs, leading to a decrease in PE utilization efficiency. This problem is particularly pronounced in layers

composed of 1×1 convolutions. From FIGURE 2, it is evident that for layers with 1×1 filters (represented by blue bars), the accelerator fails to achieve speedup benefits compared to the unpruned version.

Building upon the above observations, this paper proposes a *cross-filter structured pruning* method for layers containing 1×1 filters to enhance the processing speed of modern CNN architectures. Specifically, in layers with 1×1 convolutions, filters are clustered into groups, and corresponding channels are pruned simultaneously across all filters within a group. This pruning approach is then extended to all filter types in CNN models, based on the observation that normal $k \times k$ filters can essentially be decomposed into multiple

1×1 filters. Simulation results demonstrate that the proposed pruning method maintains comparable accuracy to the global unstructured pruning technique while significantly improving inference speed.

Additionally, this paper introduces a technique to exploit *intra-kernel parallelism* in CNNs to substantially reduce local buffer size. Traditional architectures optimized for sparsity often suffer from high local buffer memory overhead, and the proposed method addresses this limitation. The main contributions of this work are as follows:

- This paper analyzes the limitations of current hardware-algorithm co-design strategies in accelerators when processing modern CNNs.
- A new *structured pruning* approach is introduced, specifically designed to be compatible with modern CNN architectures, ensuring both high processing speed and model accuracy.
- A method for exploiting pruned data is proposed, leveraging *intra-kernel parallelism* to minimize local buffer size and enhance computational efficiency.
- The energy efficiency and area utilization of the proposed architecture are validated through FPGA implementation, while inference speed is evaluated using cycle-accurate simulators.

This paper is organized as follows: Section II provides a detailed description of the *cross-filter structured pruning* method. Section III discusses how *intra-kernel parallelism* is leveraged and gives an overview of the proposed accelerator architecture. Section IV presents the implementation details and simulation results. Finally, Section V concludes the paper.

II. CROSS-FILTER STRUCTURED PRUNING

A. LIMITATION OF UNSTRUCTURED PRUNING ON POINTWISE CONVOLUTION

As described earlier, despite pruning the networks with an overall sparsity rate of 60%, no significant speedup was observed for 1×1 convolution layers. The primary reason for this is that load imbalance becomes more severe in these layers. The commonly used pruning approach is *global unstructured pruning* [36], which removes weights below a certain threshold across the entire network. This results in an unordered distribution of zero and nonzero weights across all layers. While this method achieves the highest sparsity rate while maintaining accuracy, the irregular distribution of nonzero weights leads to varying numbers of multiplications between nonzero weights and nonzero input activations across different PEs. Consequently, during inference, PEs with fewer computations are often forced to wait for PEs with higher workloads, reducing overall efficiency.

The reason load imbalance is particularly severe in 1×1 convolution layers can be explained as follows: To minimize memory accesses, CNN accelerators typically leverage the data reuse property of CNNs, allowing data to be reused instead of repeatedly accessing global memory. Specifically, in CNNs, input activations are often shared across

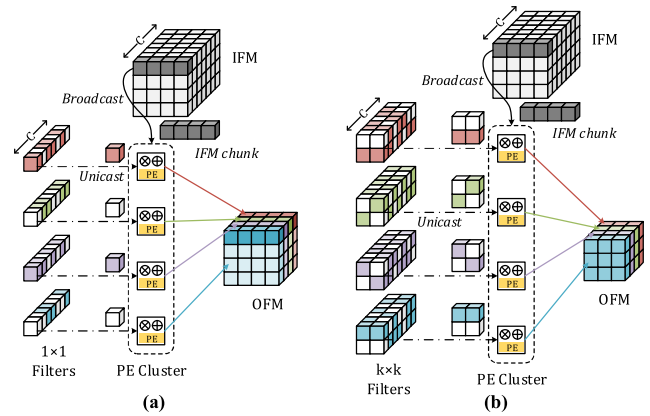


FIGURE 3. Illustration of the filter-level parallelism exploitation method with (a) pointwise convolution and (b) spatial convolution.

multiple filters, making it efficient to store weights from different filters in separate PEs and broadcast chunks of input activations to all PEs within the same PE cluster, as illustrated in FIGURE 3. Upon receiving broadcasted input activation chunks, each PE performs *multiply-accumulate (MAC)* operations independently. This method enables spatial reuse of IFMs across multiple PEs while also ensuring temporal reuse of buffered weights across different IFM chunks. Typically, a PE cluster consists of 8 to 32 PEs, a number chosen to maximize IFM reuse while maintaining an acceptable level of load imbalance across PEs.

In addition to *input activation reuse*, *convolutional reuse* must also be maximized to reduce global memory accesses. Convolutional reuse is achieved by keeping a weight stable while streaming input activations from the corresponding channel to generate *partial sums (psums)* for multiple *output activations (oacts)* within the same channel. Once the 2D convolution between a weight kernel and an input activation channel is completed, the process repeats for subsequent channels.

When a portion of input activations is broadcasted to PEs, the number of computations required per PE varies significantly. While load imbalance occurs in both $k \times k$ and 1×1 convolutions, it is more severe in 1×1 convolutions due to the way activations are distributed. This is evident in FIGURE 3(a), where input activation chunks within an IFM channel are sequentially broadcast to PEs. PEs containing nonzero weights (represented by colored boxes) remain consistently active, while PEs with zero weights (represented by white boxes) remain idle. In the worst-case scenario, if only one PE within a PE cluster contains nonzero weights, all other PEs must wait for this PE to complete computations for an entire channel before proceeding to the next channel, significantly degrading parallel efficiency.

B. CROSS-FILTER STRUCTURED PRUNING

The solution proposed is to apply *cross-filter structured pruning* to enforce a regular sparsity pattern across filters within

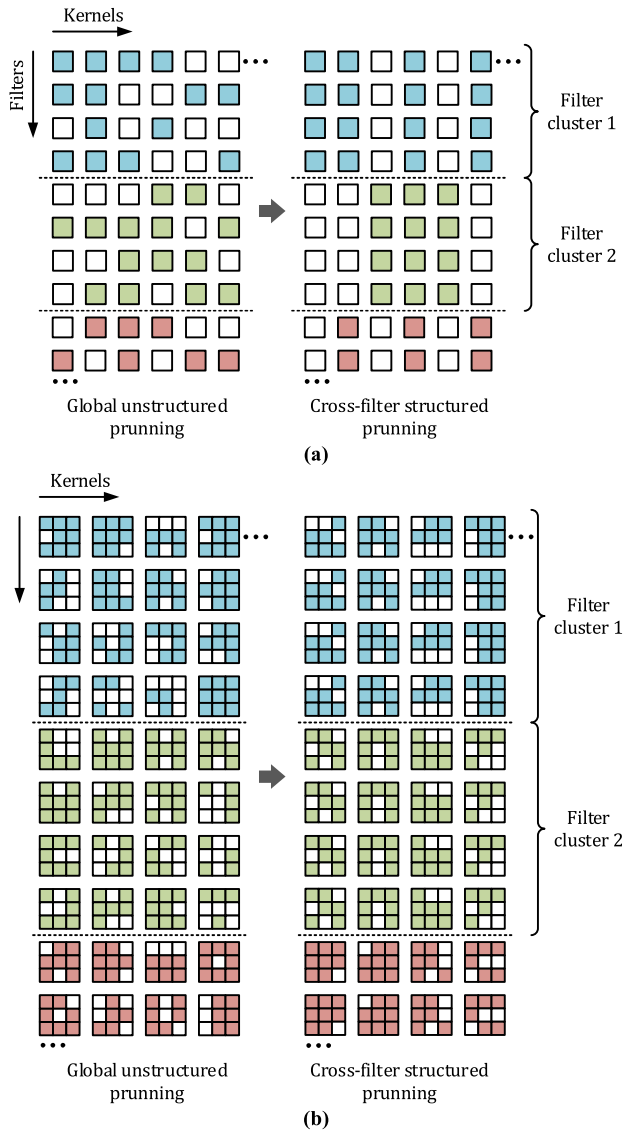


FIGURE 4. Illustration of the sparsity pattern of unstructured and structured pruning for (a) pointwise filters and (b) spatial filters.

the same PE cluster. As illustrated in FIGURE 4(a), filters in a layer are grouped into clusters, where all filters in the same cluster are stored within the PEs of a single PE cluster, with each PE processing the data of one filter. Furthermore, as shown in the figure, corresponding channels across all filters in a cluster are either entirely removed or retained. This ensures that when a channel is pruned, it is skipped uniformly across all PEs, thereby preventing load imbalance. In contrast, whenever a kernel contains nonzero weights, all PEs in the cluster execute computations simultaneously.

It is important to note that a $k \times k$ filter can be viewed as a combination of k^2 1×1 filters. Therefore, a cluster of n $k \times k$ filters can be further divided into k^2 subclusters of n 1×1 filters. The proposed *cross-filter structured pruning* method is then applied at the subcluster level as well. The result of this pruning process is illustrated in FIGURE 4(b).

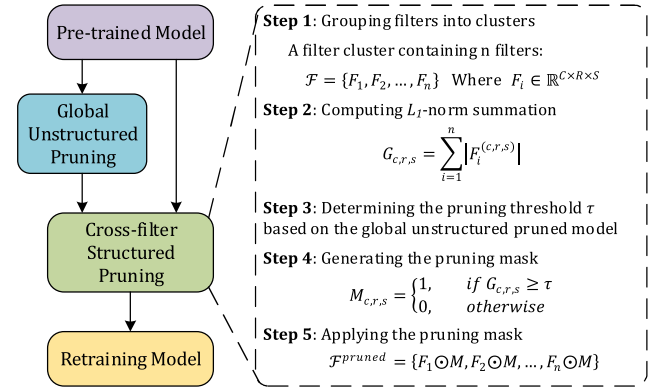


FIGURE 5. Steps for applying cross-filter structure pruning.

For 1×1 filters, entire channels within all filters in the same cluster can be pruned synchronously. For $k \times k$ filters, all filters in a cluster share a consistent kernel pattern at a channel.

The proposed pruning method offers two key advantages. First, it eliminates load imbalance across PEs by ensuring that all PEs within a cluster remain active during execution. Second, it simplifies input data handling within PEs. Since all PEs in a cluster receive the same input activations and contain weights with identical spatial positions, the output position computation for all PEs in the cluster is identical. This allows the computation process to be unified across the entire PE cluster, rather than being handled individually by each PE.

As a result, this approach eliminates the need for dedicated local *iact buffers* within each PE. Instead, all PEs within a cluster can share a common *iact buffer*, an *output calculator*, and even a *PE controller*, leading to significant hardware efficiency improvements. The key challenge of this structured pruning approach is to ensure that network accuracy degradation remains minimal compared to random pruning methods.

To achieve structured sparsity while maintaining model accuracy, a pruning process is proposed as illustrated in FIGURE 5. Since 1×1 filters are a special case of $k \times k$ filters, the figure only depicts the pruning procedure for $k \times k$ filters. The proposed pruning process consists of the following steps:

1. Training the dense model
The model is first trained in its dense form to achieve maximum accuracy.
2. Applying global unstructured pruning
Next, global unstructured pruning [36] is applied, eliminating insignificant weights across the entire model. This method has been shown to preserve accuracy while achieving the highest compression ratio. However, since weights are pruned randomly, the resulting model exhibits varying sparsity levels across different layers. These sparsity levels serve as reference values for the *cross-filter structured pruning* process.
3. Applying *cross-filter structured pruning*

The *cross-filter structured pruning* process is applied sequentially to each layer of the model. The steps for pruning a given layer are as follows:

Step 1: Grouping filters into clusters

The filters in the layer are grouped into clusters. For example, a filter cluster $\mathcal{F}$ consists of n filters:

$$\mathcal{F} = \{F_1, F_2, \dots, F_n\}$$

where each filter $F_i \in \mathbb{R}^{C \times R \times S}$ has:

- C : Number of input channels.
- R, S : Kernel height and width.

Step 2: Computing L_1 -norm summation

The absolute values of the weights at the same coordinate across all filters in the cluster are summed, forming a 3D importance matrix G :

$$G_{c,r,s} = \sum_{i=1}^n |F_i^{(c,r,s)}|$$

where:

- $c \in \{1, \dots, C\}$ iterates over input channels.
- $r \in \{1, \dots, R\}, s \in \{1, \dots, S\}$ iterate over spatial positions in the kernel.
- $i \in \{1, \dots, n\}$ iterates over the filters in the cluster.

Step 3: Determining the pruning threshold

Based on the sparsity ratio of a similar filter cluster in the randomly pruned model, a pruning threshold τ is determined for the current filter cluster.

Step 4: Generating the pruning mask

A binary mask M of the same shape as G is created, where elements are set to 1 if the corresponding value in G is greater than or equal to τ , and 0 otherwise:

$$M_{c,r,s} = \begin{cases} 1, & \text{if } G_{c,r,s} \geq \tau \\ 0, & \text{otherwise} \end{cases}$$

Step 5: Applying the pruning mask

The mask M is applied to all filters in the cluster:

$$\mathcal{F}^{\text{pruned}} = \{F_1 \odot M, F_2 \odot M, \dots, F_n \odot M\}$$

where $\odot$ represents element-wise multiplication.

4. Fine-Tuning the Pruned Model

After pruning, the pruned model is retrained to recover accuracy and optimize performance.

By applying this method, the final sparsity levels of individual layers and the entire model closely match those of the global unstructured pruned model. The key objective is to ensure that the accuracy of structuredly pruned networks remains comparable to that of unstructuredly pruned networks. Therefore, in this study, the accuracy of structuredly pruned versions is rigorously evaluated across various networks, datasets, and sparsity levels. The results, presented in FIGURE 6, demonstrate that applying *cross-filter structured pruning* alone leads to minimal accuracy degradation compared to the global unstructured

pruning approach. Specifically, experiments were conducted on several CNN architectures, including GoogLeNet [7], ResNet [8], DenseNet121 [37], and MobileNetV1 [2], using different datasets such as CIFAR-10 [38], Animals-10 [39], and ImageNette [40]. The findings indicate that the accuracy of these models remains largely unchanged, even when the proportion of pruned weights is relatively high (ranging from 50% to 90%).

C. EXISTING STRUCTURED PRUNING METHODS

Various structured pruning methods have been proposed to compress convolutional neural networks while maintaining compatibility with standard hardware. Among these, *filter-wise pruning* removes entire filters from a layer [41], and *channel-wise pruning* eliminates all channels at the same spatial location across all filters within a layer [42], [43]. These methods typically rely on different criteria to identify unimportant filters or channels. For instance, [41] employs the L_1 norm as a saliency measure to prune insignificant filters, while [42] uses the *scaling factors* in Batch Normalization (BN) layers as an important indicator to prune the preceding convolutional layer's channels. Similarly, [43] evaluates channel importance based on the *final reconstruction error* and propagates only the most influential channels. These approaches reduce model size without altering the network architecture, thus requiring no changes in hardware design while achieving reasonable inference speedup.

However, filter-wise and channel-wise pruning techniques suffer from significant accuracy degradation when applied at high sparsity levels. To address this, *group-wise structured pruning* has been introduced as a fine-grained variant of channel-wise pruning. Instead of pruning the same channel across all filters, this method first groups filters and then applies channel-wise pruning within each group. For example, the *Group-based Automatic Pruning (GAP)* algorithm [44] evaluates kernel group importance using a *Kullback-Leibler (KL)* divergence-based score, prioritizing statistically independent kernels while merging or removing redundant ones. Another method [45] uses a feature map sensitivity measure, evaluating the importance of kernel groups based on the response of an intermediate activation map to input perturbations.

Despite their advantages, group-wise pruning methods still eliminate entire channels within a filter group, which can result in notable accuracy loss at high sparsity. These methods perform relatively well on models with a large proportion of 1×1 convolutions, producing results comparable to our method. However, for architectures with fewer 1×1 convolutions, such as ResNet34, these methods exhibit significant drops in accuracy when pruning at similar sparsity levels. As illustrated in FIGURE 7, we compare the performance of global unstructured pruning, group-wise structured pruning, and our proposed cross-filter structured pruning method on ResNet34, retrained on Animals-10, CIFAR-10,

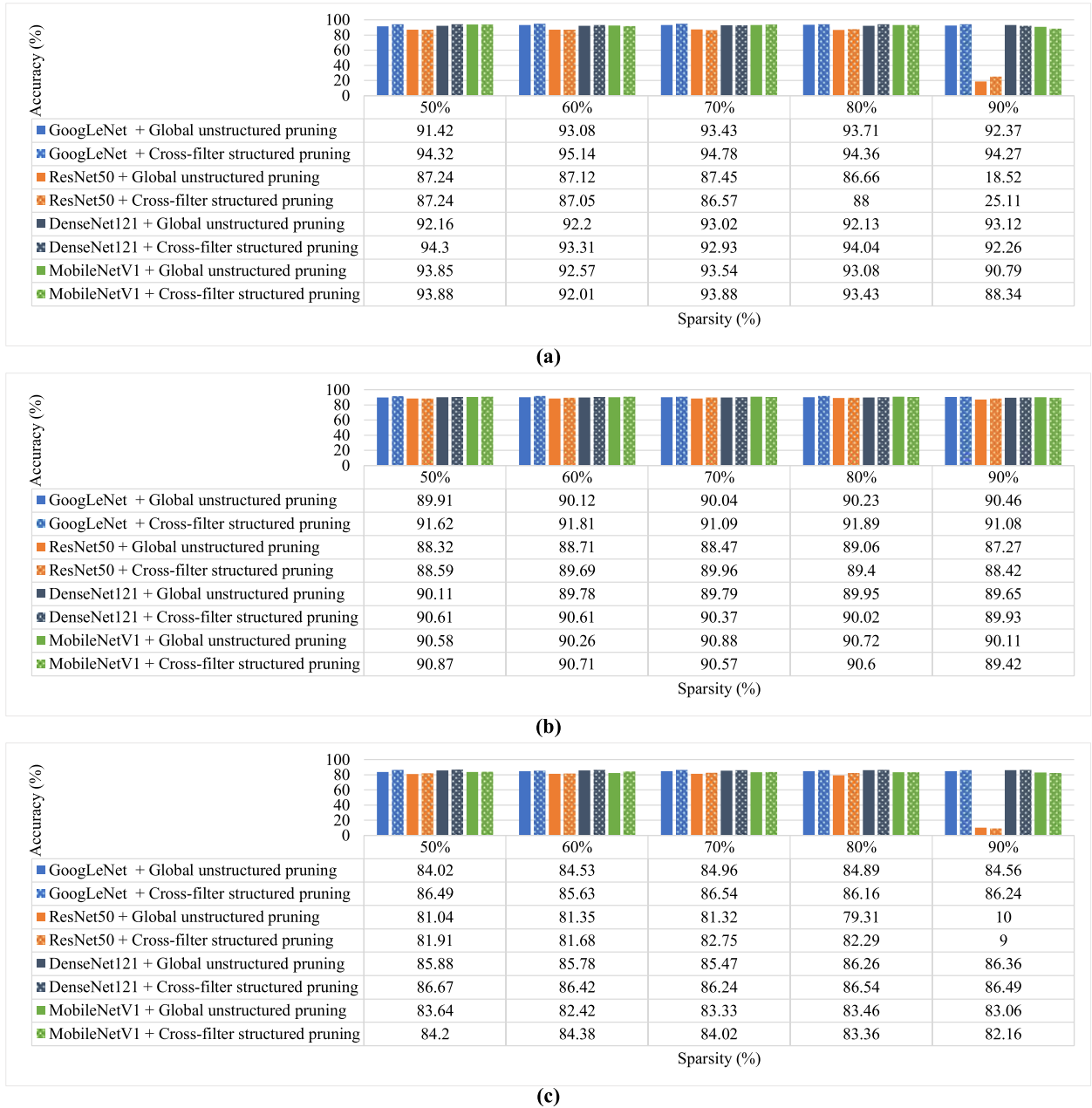


FIGURE 6. Comparison of the accuracy of models pruned with cross-filter structured pruning and global unstructured pruning for (a) Animals dataset, (b) Cifar-10 dataset, and (c) Imagenette dataset.

and ImageNette datasets. Across sparsity levels ranging from 50% to 90%, our method consistently achieves higher accuracy than group-wise structured pruning, highlighting its robustness and effectiveness across different architectures and datasets.

III. ACCELERATOR ARCHITECTURE

A. OVERALL ARCHITECTURE

FIGURE 8 illustrates the complete architecture of the proposed 8-bit integer accelerator. It consists of a *PE* array with 128 PEs, which are organized into 16 identical clusters. The compressed weight data and its metadata are

systematically partitioned and stored in an *off-chip weight DRAM*, which is structured into 16 independently operating banks. Similarly, the compressed activation data and its metadata are stored *on-chip* in an *activation SRAM*, which also consists of 16 independent banks. The SRAM has a total capacity of 1MB, which is sufficient to store intermediate outputs for layers in common CNN models.

Once the convolution operation is completed, the output data is processed through *ReLU*, *Pooling*, and *Quantization units*. The quantized 8-bit output activations are then compressed using the *CSR (Compressed Sparse Row)* format by an *output compressor* unit and stored in SRAM,

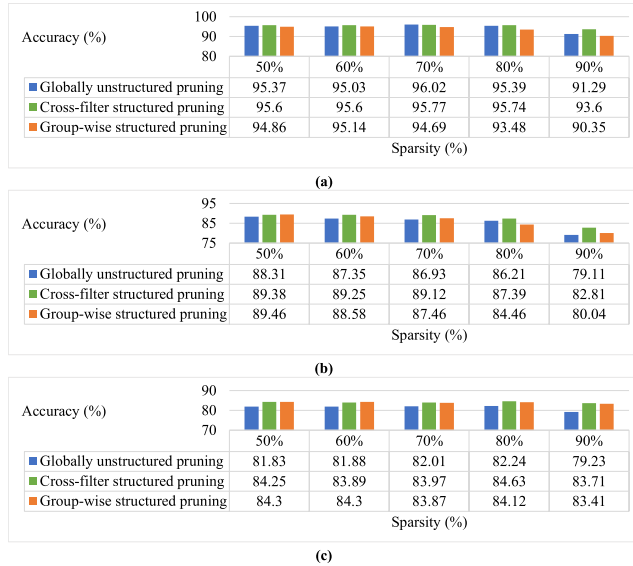


FIGURE 7. Comparison of the accuracy of ResNet34 pruned with global unstructured pruning, cross-filter structured pruning and group-wise structured pruning for (a) Animals dataset, (b) Cifar-10 dataset, and (c) Imagenette dataset.

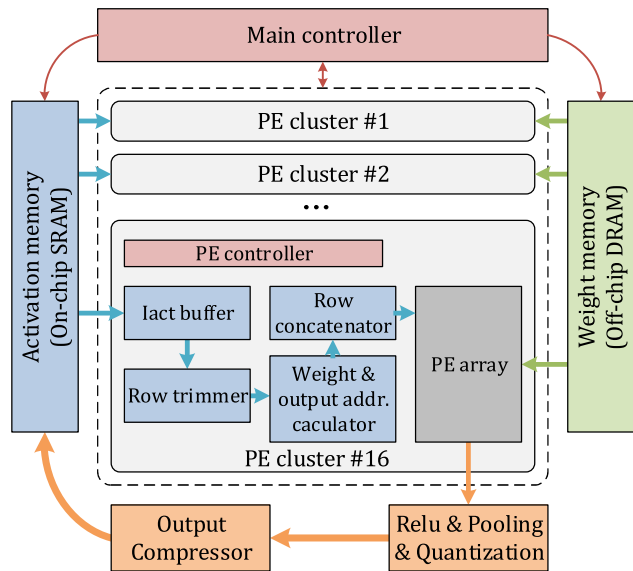


FIGURE 8. Overall architecture of the proposed accelerator.

preparing them for use as input activations in the subsequent convolution layer. All components are synchronized and managed by a *main controller*.

Each PE cluster is composed of m identical PEs, where each PE cluster is responsible for executing computations related to a single filter cluster. Within a cluster, each PE handles convolution for one filter from the filter cluster. As a result, each PE is responsible for computing either an entire output channel or a portion of an output channel. To support this, each PE is equipped with a local *weight buffer* and an *output buffer* for storing its assigned filter weights and

output activations. All PEs within a cluster share the same input activations, which are broadcast from the SRAM.

At the *multiplier* level, the accelerator adopts a *weight-stationary dataflow* strategy for performing convolution. This means that the weights remain fixed as stable inputs to the PE's multipliers, while multiple consecutive activations are sequentially processed. Due to the structured pruning applied to weights across PEs within a cluster, the skipping of invalid computations is synchronized across all PEs. Furthermore, all PEs execute operations in parallel on activations and outputs with corresponding spatial locations, which simplifies the hardware design and reduces resource overhead when implementing PE clusters. To further enhance efficiency, all PEs within a cluster share common input and output processing units, including: an *lact buffer*, a *Row trimmer*, a *Row concatenator*, and a *Weight & output address calculator*.

B. INTRA-KERNEL PARALLELISM EXPLOITATION

So far, *filter-level parallelism* has been primarily exploited, where multiple filters can be applied simultaneously to the input feature maps. The structured pruned data can also be leveraged to exploit *intra-channel parallelism*: elements within a single channel can be processed concurrently to further enhance computational efficiency. A straightforward approach to achieve this is by concurrently performing *dot products* between a weight kernel and an activation channel, a method known as *output-stationary dataflow*. However, exploiting *two-sided sparsity* using output-stationary dataflow can lead to irregular valid input pairs, which subsequently reduces PE utilization. It has been observed that intra-channel parallelism can be more effectively exploited by transforming the dataflow from output-stationary to weight-stationary dataflow, allowing multiple input pairs to be processed in parallel. With weight-stationary dataflow, input matching becomes straightforward and efficient, as all generated products are valid according to the convolution algorithm, eliminating the need for an *input matching unit*.

FIGURE 9 illustrates the function of input processors in a PE cluster. To fully utilize the processing speed of the PE array, the input activation data needs to be transformed, trimmed, and concatenated to form input vectors containing only valid input activations. The transformation of the input feature map can be accomplished by sequentially coordinating the rows read from the local activation buffer.

FIGURE 9(a) presents an example of a 2D convolution between a 4×4 input feature map and a 2×2 weight kernel, resulting in a 3×3 output feature map. For ease of data processing description, a stride of 1 is used in this example. For strides greater than 1, instead of compressing data in CSR format, the data is compressed using the *SCSR* format as introduced in [23]. In such cases, the data processing remains identical to the stride = 1 case. Although different PEs process different weight kernels, the masks of these weight kernels remain identical. The CSR-compressed representations of the input activations and weights are shown

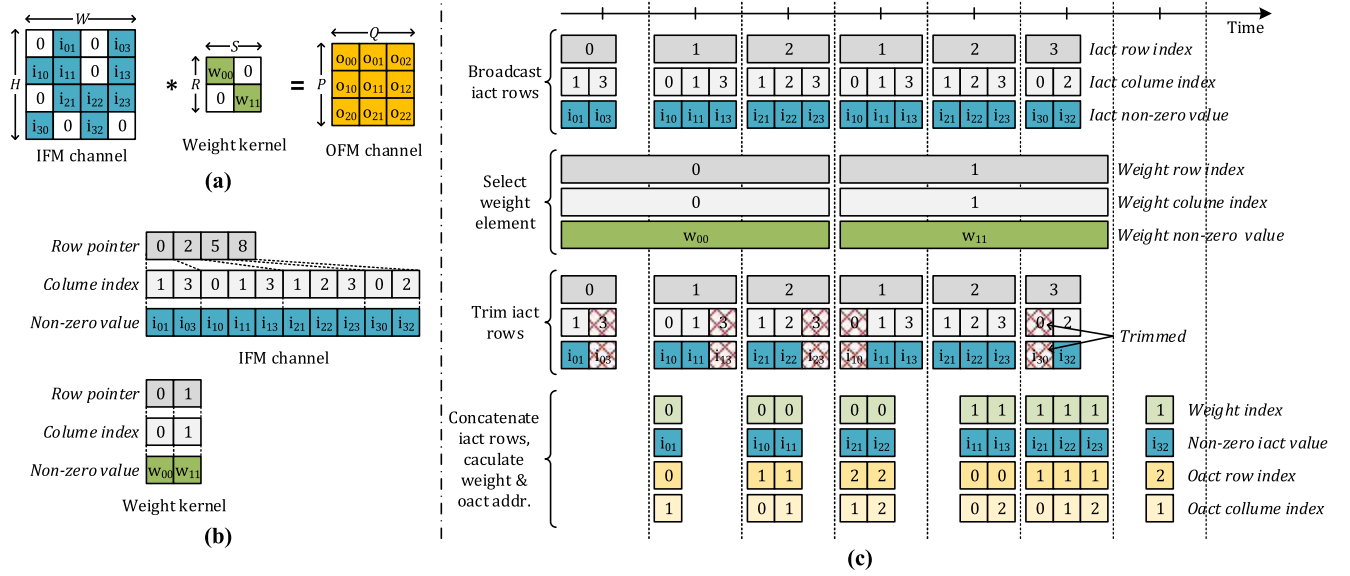


FIGURE 9. (a) An example of spatial convolution, (b) Input data compressed with CSR format, (c) Illustration of the processing of input activations at a PE cluster.

in FIGURE 9(b). The compressed input activation data is stored in activation SRAM, while the compressed weight data is stored in weight DRAM. Due to the structured pruning of weights, the compressed weights within a cluster share a common row pointer array and column index array. The compressed weights and their metadata are then unicast to the respective PEs, while the compressed input activations and their metadata are broadcast to all PEs within the cluster.

Under the weight-stationary dataflow, each stored weight in a PE's weight buffer is sequentially selected. Then, each corresponding row of the input activation that can participate in convolution with the selected weight is read sequentially from the activation buffer for processing. FIGURE 9(c) illustrates the broadcast, trimming, and concatenation of activation rows. For example, if weight w_{00} is selected, only rows 0, 1, and 2 of the IFM are broadcast, as they contribute to valid multiplications. The formula for selecting a valid row index h is as follows:

$$r \leq h \leq H - R + r$$

where r is the row index of the selected weight, h is the row index of the IFM matrix, H is the height of the IFM matrix, and R is the height of the weight matrix. In the given example, when w_{00} with $r = 0$ is selected, row index $h = 3$ is not broadcast. The selection of rows to be broadcast is managed by the PE controller.

Once each activation row is broadcast, some activation values are trimmed based on the position of the stationary weight, as certain activation values will not contribute to meaningful products. The formula for determining valid activation column indices is:

$$s \leq w \leq W - S + s$$

where s is the column index of the selected weight, w is the column index of the activation, W is the width of the IFM matrix, and S is the width of the weight matrix. In the given example, when w_{00} with $s = 0$ is selected, activation column indices $w = 3$ (i_{03} , i_{13} , i_{23}) are trimmed. The Row trimmer synchronously trims multiple invalid activation elements within a single row.

After trimming, the corresponding weight addresses and output addresses for each activation element are synchronously processed by the Weight & output address calculator. Subsequently, the activation rows are concatenated using the Row concatenator. The weight index represents the weight that an activation element will be multiplied with. The output row index p and output column index q are computed as follows:

$$p = h - r, \quad q = w - s$$

The processed data is then fed into the PE array. The PE array performs multiple MAC operations in parallel on these activation elements. Since all activation elements contribute to valid operations, the PEs can operate continuously without interruption, enhancing the accelerator's processing speed.

C. PE ARCHITECTURE

The structure of a PE with m MACs is illustrated in FIGURE 10. As previously described, each PE is responsible for computing one output channel or a portion of an output channel. Consequently, each PE includes an oact buffer to store accumulated partial sum values. The oact buffer is designed with a size of $14 \times 14 \times 24$ -bit. Since each PE processes a separate channel, a dedicated weight buffer is allocated for each PE. This buffer stores the entire kernel

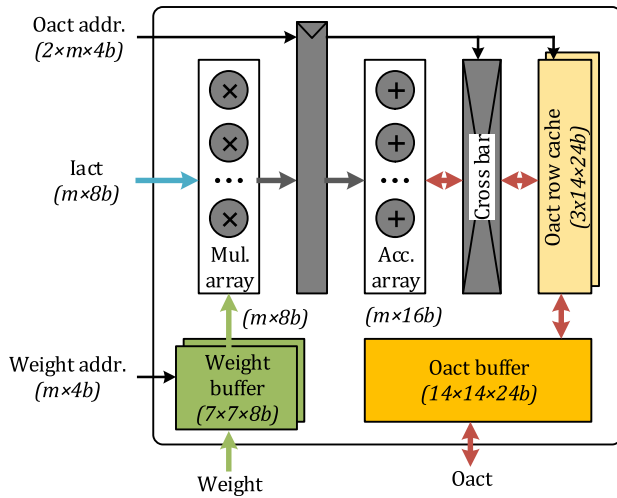


FIGURE 10. Architecture of a PE with n MACs.

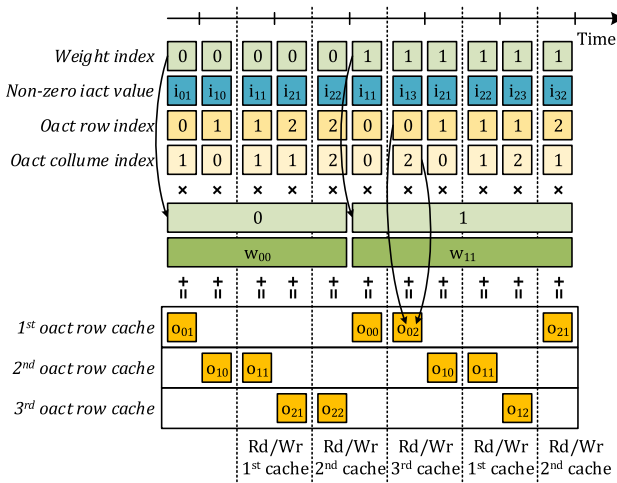


FIGURE 11. Illustration of data process of a PE with 2 parallel MACs.

of a filter, with a capacity of $7 \times 7 \times 8$ -bit in this design. These weights participate in convolution with the broadcasted activation vectors. Weight data is updated only when the corresponding weight kernel completes all necessary computations, at which point convolution proceeds to the next set of channels. To mitigate delays caused by memory read latency, the weight buffer is doubled.

FIGURE 11 illustrates the operation on the input data of a PE with $m = 2$ MACs operating in parallel. In each clock cycle, 2 activation elements are synchronously read and processed by the multiplier array. Each activation element carries information about its corresponding weight and output, ensuring that the appropriate weight value is retrieved from the weight buffer and supplied to the multiplier array for computation. Simultaneously, output addresses are used to select the necessary outputs for accumulation. Since activation values are read in a row-wise manner, the corresponding output values are also generated in a row-wise sequence.

To reduce energy consumption related to *output buffer* accesses, three rows of the output buffer are cached. The cached output values directly accumulate with the output products from the multiplier array. When a cached row of output is fully accumulated, the row is written back to the output buffer, and a subsequent row is read into the cache. A crossbar is required to route output activations to the corresponding accumulators. By adopting the weight-stationary dataflow, each generated output product has a unique memory address, preventing congestion and simplifying the design of the output crossbar.

D. FULLY-CONNECTED LAYER

Fully-connected layers are a special case of convolutional layers, where the weight filter size matches the input feature map size, and the filter does not convolve with the input feature map. As a result, convolutional data reuse cannot be exploited for fully-connected layers. FIGURE 12 illustrates the operation for a fully-connected layer. Similar to convolutional layers, in fully-connected layers, the IFM also participates in dot products with multiple filters. Therefore, the hardware architecture used in this paper can be directly reused for processing fully-connected layers. Since convolutional data reuse is not feasible, weight-stationary dataflow is replaced by output-stationary dataflow for fully-connected layers. Each weight filter performs a dot product with the IFM to generate a single output activation, as shown in FIGURE 12 (a).

The weights in these layers are pruned unstructuredly and remain *uncompressed*, whereas the activation data is *compressed* and broadcast to preprocessing units to compute valid *activation-weight pairs*, as shown in FIGURE 12 (b). Consequently, in these layers, only *one-sided sparsity* (activation sparsity) is exploited. For activation-weight pairs, whenever a weight value is zero, the corresponding multiplier is simply turned off to reduce power consumption, rather than being skipped, as in convolution layers. With this design, hardware modifications are minimal. The only significant change in the PE hardware for these layers is reconfiguring the accumulator array into an *adder tree*, as shown in FIGURE 12 (c). With n inputs from the n accumulators in the PE, only $n - 1$ adders are needed to form an *adder tree*. Therefore, the existing accumulators are sufficient to be reconnected to form an *adder tree* without the need to add more hardware.

IV. EVALUATION

A. SPEEDUP COMPARISON

Developing a complete accelerator and evaluating its performance using ASIC or FPGA implementations typically requires substantial time and effort. Therefore, to estimate the speedup of the proposed architecture, a *cycle-accurate simulator* was implemented in *Python*. This simulator executed inference tasks on real-world models and datasets, recording the total number of clock cycles required to process each layer and the entire model. The recorded execution cycles were

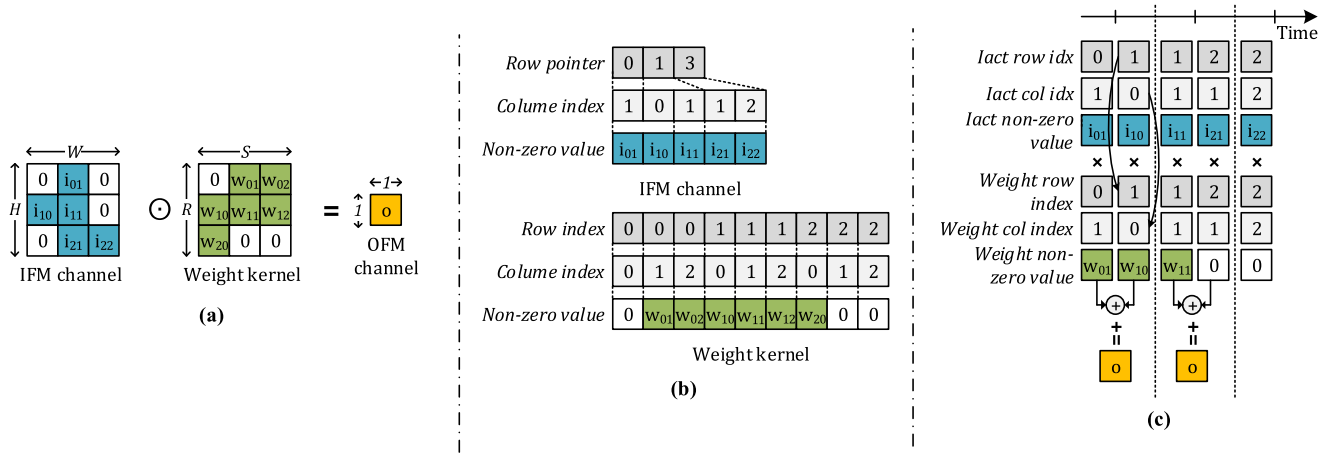


FIGURE 12. (a) An example of a fully-connected layer, (b) How data is stored in memory, (c) Execution of data process of the fully-connected layer.

then compared with the inference time of the same models on other architectures. The selected comparison architectures included state-of-the-art accelerators such as GoSPA [19], Sparten [18], and StarSPA [23]. Additionally, a dense architecture was employed as a baseline to assess speedup across different designs. All architectures were configured with an equivalent computational capacity of 1024 MAC units.

As shown in FIGURE 6, when the number of processing elements in a PE cluster was set to $n = 8$, the accuracy obtained using the *cross-filter structured pruning* method exhibited minimal deviation compared to that of the global unstructured pruning method. Consequently, a PE cluster size of $n = 8$ was selected for the cycle-accurate simulator design of the proposed architecture. Additionally, the number of MAC units per PE in the proposed architecture was set to $m = 8$. As a result, the total number of PEs in the proposed architecture was determined as $1024/(8 \times 8) = 16$. In contrast, the number of PEs in the other accelerators was $1024/8 = 128$, since in these designs, each PE contained only a single MAC unit.

These architectures executed inference on models including ResNet34 [8], GoogLeNet [7], ResNet50 [8], DenseNet121 [37], MobileNetV1 [2], and EfficientNetV2_m [35], with the corresponding clock cycles recorded. The selected models were pruned to an approximate sparsity level of 60% using cross-filter structured pruning and global unstructured pruning, followed by fine-tuning to restore accuracy to levels comparable to their unpruned versions. The proposed architecture performed inference on models pruned using the cross-filter structured pruning method, whereas the comparison architectures executed inference on models pruned using the global unstructured pruning method.

FIGURE 13 illustrates the speedup results of various architectures, normalized against the performance of the dense architecture. Across all evaluated models, the proposed architecture consistently achieved higher speed than state-of-the-art architectures. Specifically, the proposed

architecture was 1.6, 1.24, 1.14, 1.23, 1.25, and 1.15 times faster than Sparten for ResNet34 [8], GoogLeNet [7], ResNet50 [8], DenseNet121 [37], MobileNetV1 [2], and EfficientNetV2_m [35], respectively. This speed improvement can be attributed to the structured pruning of network filters, which enabled the synchronous skipping of invalid computations across PEs, effectively eliminating load imbalance.

B. AREA EFFICIENCY EVALUATION

To evaluate the hardware resource requirements for implementing the proposed accelerator, a PE cluster was designed in Verilog and synthesized on an FPGA Zynq-Ultrascale+ using Vivado [46]. The design was optimized to operate reliably at 100 MHz. Instead of utilizing built-in DSP blocks, all MAC units were implemented using *lookup tables (LUTs)* and *flip-flops (FFs)*. For a comparative assessment of hardware resource utilization, compute clusters with an equivalent number of MACs (64 MACs) from Sparten [18] and StarSPA [23] were also synthesized. After synthesis, the LUT and FF counts required for each PE cluster were reported in TABLE 2.

The results indicate that the proposed architecture achieved the lowest utilization of LUTs and FFs among the evaluated designs. Specifically, compared to Sparten [18], the proposed design reduced LUT and FF usage by factors of $7.85\times$ and $10.95\times$, respectively. Similarly, compared to StarSPA [23], LUT and FF usage were reduced by factors of $1.73\times$ and $6.31\times$, respectively. This reduction was primarily attributed to the structured pruning approach, which simplified input processing. Within the proposed architecture, PEs in the same cluster shared input and output processing units, including the *lact buffer*, *Row trimmer*, *Row concatenator*, *Weight & output address calculator*, and *PE controller*. Additionally, each PE integrated multiple *MAC units* (8 per PE), which shared *input* and *output buffers*, significantly reducing flip-flop utilization. Although the proposed design incorporated a $3 \times 14 \times 24$ -bit *output cache* between the *accumulator array* and the



FIGURE 13. Comparisons on normalized speedup on convolution layers of (a) ResNet34, (b) GoogLeNet, (c) ResNet50, (d) DenseNet121, (e) MobileNetV1 and (f) EfficientNetV2_m (baseline: Dense architecture).

output buffer, the overall flip-flop usage remained significantly lower than that of StarSPA. Furthermore, the inclusion of the *output cache* reduced the *crossbar* size, thereby lowering power consumption associated with read and write operations to the *output buffer*.

C. ENERGY EFFICIENCY EVALUATION

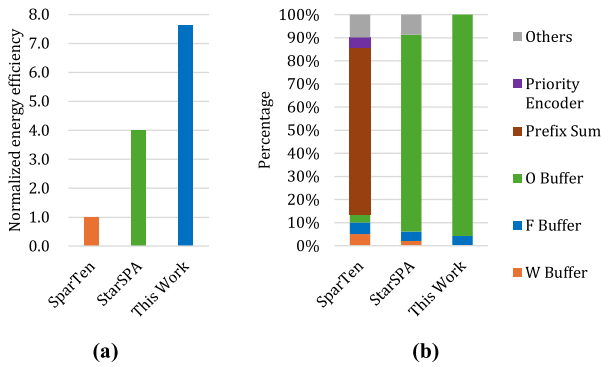
To evaluate the energy efficiency of the proposed design, Verilog implementations were simulated using synthetic input data. Specifically, a $16 \times 16 \times 196$ input feature map was convolved with 196 filters of size $3 \times 3 \times 196$,

producing an output feature map of size $14 \times 14 \times 196$. Both the input feature map and filters were generated with a sparsity ratio of 60%. This synthetic workload was simulated on the synthesized designs of the previously discussed accelerators. The clock frequency was set to 100 MHz, and switching signals were collected to generate power reports for each design. The total energy consumption was calculated by multiplying the reported power consumption with the execution time required to process the input workload.

The normalized energy results are shown in FIGURE 14. Compared to SparTen [18] and StarSPA [23], the proposed

TABLE 2. Numbers of FPGA primitives in synthesized PE clusters.

FPGA primitives	Breakdown of FPGA primitives	Sparten [18]	StarSPA [23]	This Work
Number of LUTs	Prefix Sum	200,320	0	0
	Priority Encoder	193,408	0	0
	Weight Buffer	121,856	22,578	8,408
	Iact Buffer	60,928	13,696	1,916
	Oact Buffer	59,685	111,216	74,414
	Others	64,884	6,712	4,563
	Total	701,081	154,202	89,302
Number of FF	Prefix Sum	1,152	0	0
	Priority Encoder	0	0	0
	Weight Buffer	295,424	37,156	9,289
	Iact Buffer	147,712	26,240	7,072
	Oact Buffer	196,639	289,776	43,013
	Others	9,728	21,824	20
	Total	650,655	374,996	59,394

**FIGURE 14.** Energy consumption on Zynq-Ultrascale+ FPGA (a) Normalized energy efficiency and (b) Energy breakdown.

design achieved $7.6\times$ and $1.9\times$ higher energy efficiency, respectively. A detailed analysis revealed that, while SparTen's most power-intensive component was the *inner-join logic*, particularly the *prefix sum* computations, the dominant power consumer in both StarSPA and the proposed architecture was the *output buffer*. This was attributed to the weight-stationary dataflow employed by both StarSPA and the proposed design, which resulted in frequent accesses to the *output buffer*. However, by integrating an *output cache*, the proposed design prevented the accumulator array from directly accessing the *output buffer*, thereby significantly reducing its power consumption. Furthermore, synchronizing the operations across PEs within a cluster minimized the number of required input processing units, leading to further energy savings.

V. CONCLUSION

This paper proposes a structured pruning technique named *Cross-filter structured pruning*, aimed at optimizing CNN inference speed, especially in light of the growing use

of pointwise convolutions. The proposed pruning method ensures synchronized processing across processing elements while maintaining high model accuracy. Building upon this pruned input data, the paper introduces a novel accelerator architecture that effectively utilizes input sparsity to accelerate inference speed, reduce hardware complexity, and lower computational energy consumption. Experimental results underscore the importance of algorithm-hardware co-design in optimizing CNN inference efficiency. The compact design of the proposed architecture makes it ideal for energy-efficient applications that require real-time processing. Additionally, it remains compatible with both structured and unstructured pruning methods, as well as unpruned models, making it a versatile solution adaptable to a wide range of CNN architectures and applications.

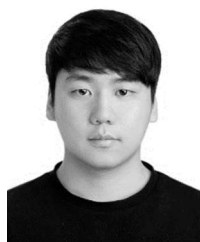
REFERENCES

- [1] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, "Swin transformer: Hierarchical vision transformer using shifted windows," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 9992–10002, doi: [10.1109/ICCV48922.2021.00986](https://doi.org/10.1109/ICCV48922.2021.00986).
- [2] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, "MobileNets: Efficient convolutional neural networks for mobile vision applications," 2017, *arXiv:1704.04861*.
- [3] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 779–788.
- [4] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, 2015, doi: [10.1038/nature14539](https://doi.org/10.1038/nature14539).
- [5] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2014, *arXiv:1409.1556*.
- [6] M. Lin, Q. Chen, and S. Yan, "Network in network," 2013, *arXiv:1312.4400*.
- [7] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, "Going deeper with convolutions," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2015, pp. 1–9.
- [8] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.
- [9] X. Zhang, X. Zhou, M. Lin, and J. Sun, "ShuffleNet: An extremely efficient convolutional neural network for mobile devices," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 6848–6856.
- [10] J. Albericio, P. Judd, T. Hetherington, T. Aamodt, N. E. Jerger, and A. Moshovos, "Cnvlutin: Ineffectual-neuron-free deep neural network computing," presented at the *Proc. ACM/IEEE 43rd Annu. Int. Symp. Comput. Archit. (ISCA)*, Jun. 2016, doi: [10.1109/ISCA.2016.11](https://doi.org/10.1109/ISCA.2016.11).
- [11] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," *IEEE J. Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, Jan. 2017.
- [12] A. Aimar, H. Mostafa, E. Calabrese, A. Rios-Navarro, R. Tapiador-Morales, I.-A. Lungu, M. B. Milde, F. Corradi, A. Linares-Barranco, S.-C. Liu, and T. Delbruck, "NullHop: A flexible convolutional neural network accelerator based on sparse representations of feature maps," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 30, no. 3, pp. 644–656, Mar. 2019, doi: [10.1109/TNNLS.2018.2852335](https://doi.org/10.1109/TNNLS.2018.2852335).
- [13] S. Zhang, Z. Du, L. Zhang, H. Lan, S. Liu, L. Li, Q. Guo, T. Chen, and Y. Chen, "Cambricon-X: An accelerator for sparse neural networks," in *Proc. 49th Annu. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, Oct. 2016, pp. 1–12, doi: [10.1109/MICRO.2016.7783723](https://doi.org/10.1109/MICRO.2016.7783723).

- [14] N. Srivastava, H. Jin, S. Smith, H. Rong, D. Albonese, and Z. Zhang, "Tensaurus: A versatile accelerator for mixed sparse-dense tensor computations," in *Proc. IEEE Int. Symp. High Perform. Comput. Archit. (HPCA)*, Feb. 2020, pp. 689–702, doi: [10.1109/HPCA47549.2020.00062](https://doi.org/10.1109/HPCA47549.2020.00062).
- [15] E. Qin, A. Samajdar, H. Kwon, V. Nadella, S. Srinivasan, D. Das, B. Kaul, and T. Krishna, "SIGMA: A sparse and irregular GEMM accelerator with flexible interconnects for DNN training," in *Proc. IEEE Int. Symp. High Perform. Comput. Archit. (HPCA)*, Feb. 2020, pp. 58–70, doi: [10.1109/HPCA47549.2020.00015](https://doi.org/10.1109/HPCA47549.2020.00015).
- [16] Y.-H. Chen, T.-J. Yang, J. Emer, and V. Sze, "Eyeriss v2: A flexible accelerator for emerging deep neural networks on mobile devices," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 9, no. 2, pp. 292–308, Jun. 2019, doi: [10.1109/JETCAS.2019.2910232](https://doi.org/10.1109/JETCAS.2019.2910232).
- [17] A. Parashar, M. Rhu, A. Mukkara, A. Puglielli, R. Venkatesan, B. Khailany, J. Emer, S. W. Keckler, and W. J. Dally, "SCNN: An accelerator for compressed-sparse convolutional neural networks," in *Proc. ACM/IEEE 44th Annu. Int. Symp. Comput. Archit. (ISCA)*, Jun. 2017, pp. 27–40, doi: [10.1145/3079856.3080254](https://doi.org/10.1145/3079856.3080254).
- [18] A. Gondimalla, N. Chesnut, M. Thottethodi, and T. N. Vijaykumar, "SparTen: A sparse tensor accelerator for convolutional neural networks," presented at the *Proc. 52nd Annu. IEEE/ACM Int. Symp. Microarchitecture*, Columbus, OH, USA, Oct. 2019, doi: [10.1145/3352460.3358291](https://doi.org/10.1145/3352460.3358291).
- [19] C. Deng, Y. Sui, S. Liao, X. Qian, and B. Yuan, "GoSPA: An energy-efficient high-performance globally optimized SParse convolutional neural network accelerator," in *Proc. ACM/IEEE 48th Annu. Int. Symp. Comput. Archit. (ISCA)*, Jun. 2021, pp. 1110–1123, doi: [10.1109/ISCA52012.2021.00090](https://doi.org/10.1109/ISCA52012.2021.00090).
- [20] M. A. Qureshi and A. Munir, "Sparse-PE: A performance-efficient processing engine core for sparse convolutional neural networks," *IEEE Access*, vol. 9, pp. 151458–151475, 2021, doi: [10.1109/ACCESS.2021.3126708](https://doi.org/10.1109/ACCESS.2021.3126708).
- [21] J. Yang, W. Fu, X. Cheng, X. Ye, P. Dai, and W. Zhao, "S² engine: A novel systolic architecture for sparse convolutional neural networks," *IEEE Trans. Comput.*, vol. 71, no. 6, pp. 1440–1452, Jun. 2022, doi: [10.1109/TC.2021.3087946](https://doi.org/10.1109/TC.2021.3087946).
- [22] N.-S. Pham and T. Suh, "Optimization of microarchitecture and dataflow for sparse tensor CNN acceleration," *IEEE Access*, vol. 11, pp. 108818–108832, 2023, doi: [10.1109/ACCESS.2023.3319727](https://doi.org/10.1109/ACCESS.2023.3319727).
- [23] N.-S. Pham, S. Shin, L. Xu, W. Shi, and T. Suh, "StarSPA: Stride-aware sparsity compression for efficient CNN acceleration," *IEEE Access*, vol. 12, pp. 10893–10909, 2024, doi: [10.1109/ACCESS.2024.3353313](https://doi.org/10.1109/ACCESS.2024.3353313).
- [24] Z.-G. Liu, P. N. Whatmough, Y. Zhu, and M. Mattina, "S2TA: Exploiting structured sparsity for energy-efficient mobile CNN acceleration," presented at the *Proc. IEEE Int. Symp. High-Perform. Comput. Archit. (HPCA)*, Apr. 2022, doi: [10.1109/HPCA53966.2022.00049](https://doi.org/10.1109/HPCA53966.2022.00049).
- [25] A. Ali, I. Ullah, S. K. Singh, A. Sharafian, W. Jiang, H. I. Sherazi, and X. Bai, "Energy-efficient resource allocation for urban traffic flow prediction in edge-cloud computing," *Int. J. Intell. Syst.*, vol. 2025, no. 1, pp. 1–19, Jan. 2025, doi: [10.1155/int/1863025](https://doi.org/10.1155/int/1863025).
- [26] A. Ali, I. Ullah, S. Ahmad, Z. Wu, J. Li, and X. Bai, "An attention-driven spatio-temporal deep hybrid neural networks for traffic flow prediction in transportation systems," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 4, pp. 1–15, Apr. 2025, doi: [10.1109/TITS.2025.3540852](https://doi.org/10.1109/TITS.2025.3540852).
- [27] H. Hussain, M. Zakarya, A. Ali, A. A. Khan, M. R. C. Qazani, M. Al-Bahri, and M. Haleem, "Energy efficient real-time tasks scheduling on high-performance edge-computing systems using genetic algorithm," *IEEE Access*, vol. 12, pp. 54879–54892, 2024, doi: [10.1109/ACCESS.2024.3388837](https://doi.org/10.1109/ACCESS.2024.3388837).
- [28] A. Ali, I. Ullah, M. Shabaz, A. Sharafian, M. A. Khan, X. Bai, and L. Qiu, "A resource-aware multi-graph neural network for urban traffic flow prediction in multi-access edge computing systems," *IEEE Trans. Consum. Electron.*, vol. 70, no. 4, pp. 7252–7265, Nov. 2024, doi: [10.1109/TCE.2024.3439719](https://doi.org/10.1109/TCE.2024.3439719).
- [29] I. Ullah, D. Adhikari, F. Ali, A. Ali, H. Khan, A. Sharafian, S. M. Kesavan, and X. Bai, "Revolutionizing e-commerce with consumer-driven energy-efficient WSNs: A multi-characteristics approach," *IEEE Trans. Consum. Electron.*, vol. 70, no. 4, pp. 6871–6882, Nov. 2024, doi: [10.1109/TCE.2024.3411606](https://doi.org/10.1109/TCE.2024.3411606).
- [30] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding," 2015, *arXiv:1510.00149*.
- [31] M. Horowitz, "1.1 computing's energy problem (and what we can do about it)," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2014, pp. 10–14.
- [32] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, "Efficient processing of deep neural networks: A tutorial and survey," *Proc. IEEE*, vol. 105, no. 12, pp. 2295–2329, Dec. 2017.
- [33] V. Nair and G. E. Hinton, "Rectified linear units improve restricted Boltzmann machines," presented at the 27th Int. Conf. Int. Conf. Mach. Learn., 2010.
- [34] S. Kang, G. Park, S. Kim, S. Kim, D. Han, and H.-J. Yoo, "An overview of sparsity exploitation in CNNs for on-device intelligence with software-hardware cross-layer optimizations," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 11, no. 4, pp. 634–648, Dec. 2021, doi: [10.1109/JETCAS.2021.3120417](https://doi.org/10.1109/JETCAS.2021.3120417).
- [35] M. Tan and Q. V. Le, "EfficientNetV2: Smaller models and faster training," in *Proc. Int. Conf. Mach. Learn.*, Jan. 2021, pp. 10096–10106.
- [36] M. Paganini, *Pruning Tutorial*. Accessed: Mar. 11, 2025. [Online]. Available: https://pytorch.org/tutorials/intermediate/pruning_tutorial.html
- [37] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 2261–2269.
- [38] V. N. A. Krizhevsky, Geoffrey Hinton. *The CIFAR-10 Dataset*. Accessed: Jan. 8, 2025. [Online]. Available: <http://www.cs.toronto.edu/kriz/cifar.html>
- [39] A. Corrado. *Animals-10 Dataset*. Accessed: Dec. 15, 2024. [Online]. Available: <https://www.kaggle.com/datasets/alessiocrorad099/animals10>
- [40] J. Howard. *ImageNet: A Smaller Subset of 10 Easily Classified Classes From Imagenet*. Accessed: Jan. 16, 2025. [Online]. Available: <https://github.com/fastai/imagenette>
- [41] H. Li, A. Kadav, I. Durdanovic, H. Samet, and H. P. Graf, "Pruning filters for efficient ConvNets," 2016, *arXiv:1608.08710*.
- [42] Z. Liu, J. Li, Z. Shen, G. Huang, S. Yan, and C. Zhang, "Learning efficient convolutional networks through network slimming," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 2755–2763.
- [43] R. Yu, A. Li, C.-F. Chen, J.-H. Lai, V. I. Morariu, X. Han, M. Gao, C.-Y. Lin, and L. S. Davis, "NISIP: Pruning networks using neuron importance score propagation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 9194–9203.
- [44] D. Chen, K. Lin, and Q. Deng, "GAP: A group-based automatic pruning algorithm via convolution kernel fusion," *Neurocomputing*, vol. 609, Dec. 2024, Art. no. 128488, doi: [10.1016/j.neucom.2024.128488](https://doi.org/10.1016/j.neucom.2024.128488).
- [45] N. Yu, S. Qiu, X. Hu, and J. Li, "Accelerating convolutional neural networks by group-wise 2D-filter pruning," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, May 2017, pp. 2502–2509, doi: [10.1109/IJCNN.2017.7966160](https://doi.org/10.1109/IJCNN.2017.7966160).
- [46] Vivado. (2021). *Xilinx, Inc.* [Online]. Available: <https://www.xilinx.com/support/download.html>



NGOC-SON PHAM (Member, IEEE) received the B.S. degree from Vietnam National University, Hanoi, Vietnam, in 2011, and the M.S. and Ph.D. degrees in electrical and electronics engineering from Chung-Ang University, Seoul, South Korea, in 2019. He is currently a Research Professor with the Department of Computer Science and Engineering, Korea University. His research interests include mixed-signal integrated circuit design, embedded and real-time systems, machine learning accelerators, and security.



SANGWON SHIN received the B.S. degree in computer science and engineering from Korea University, Seoul, South Korea, in 2023, where he is currently pursuing the Ph.D. degree with the Department of Computer Science and Engineering. His research interests include hardware architecture, security, and cryptography.



WEIDONG SHI (Senior Member, IEEE) is currently an Associate Professor with the University of Houston. His research interests include high-performance computer architecture for large-scale multimedia applications, applied security, real-time computer graphics, blockchain, and cloud computing.



LEIXU (Member, IEEE) received the Ph.D. degree from the Chinese Academy of Sciences, China. He is currently an Assistant Professor with the Department of Computer Science, Kent State University. His research interests include applied cryptography, cloud/mobile security, and decentralized systems.



TAEWEON SUH (Member, IEEE) received the B.S. degree in electrical engineering from Korea University, Seoul, South Korea, in 1993, the M.S. degree in electronics engineering from Seoul National University, in 1995, and the Ph.D. degree in electrical and computer engineering from Georgia Institute of Technology, Atlanta, GA, USA, in 2006. He is currently a Professor with the Department of Computer Science and Engineering, Korea University.

...